Predicting Term Deposit Subscription Using the UCI Bank Marketing Dataset

Students: Naresh Rambhatla & Rahul Sajith
Institution: Michigan Technological University
Date: December 2025

Table of Contents

Table of Contents 1
1. Introduction 1
2. Data and Initial Cleaning 1
2.1 Dataset Source and Description 1
2.2 Loading the Data and First Look 2
2.3 Removing the Leakage Variable duration 4
2.4 Handling Missing Values and "unknown" Levels 5
3. Feature Engineering and Preprocessing 6
3.1 Special Handling of pdays 6
3.2 Numeric Variables: Skewness, Transforms, and Correlation 9
Numeric NZV filtering 11
Numeric correlation filtering 12
3.3 Outlier Handling and Spatial Sign + Center/Scale 13
3.4 Categorical Variables and Dummy Encoding 15
3.5 NZV and Correlation Filtering on Dummies 16
3.6 Final Design Matrix and Predictor Count 19
4. Train–Test Split and Class Imbalance 19
4.1 Stratified Train–Test Split 19
4.2 Class Imbalance and Implications 20
5. Resampling Strategy, SMOTE, and Evaluation Metric 20
5.1 Repeated Cross-Validation Setup 20
5.2 Why ROC/AUC (and not just Accuracy or Kappa)? 21
5.3 Why SMOTE, and Where It Was Used 22
6. Models Fit and Tuning Decisions 23
6.1 Baseline Logistic Regression (GLM) 24
6.2 Penalized Logistic Regression (GLMNet) 25
6.3 Flexible Discriminant Analysis (FDA) 27
6.4 Neural Network (Single Hidden Layer) 28
6.5 Partial Least Squares Discriminant Analysis (PLSDA) 30
6.6 Linear Discriminant Analysis (LDA) 32
6.7 k-Nearest Neighbors (KNN) 33
6.8 Attempted SVM and Other Alternatives 34
7. Cross-Validated Model Comparison 35
8. Test-Set Performance and Confusion Matrices 37
8.1 Overall Test-Set Comparison Table 37
8.2 Logistic Regression (Final Reference Model) 38
8.3 Other Models: Test Behavior 40

9. Probability Threshold Tuning..... 42
9.1 Logistic Regression Threshold Sweep..... 42
9.2 Linking Thresholds to Business Trade-Offs..... 44
10. Final Model Choice and Interpretation..... 44
11. Limitations and Future Work..... 46
12. Conclusions..... 46
References..... 47
Appendix A – Test-Set Performance Table..... 47
Appendix B – ROC Curves and Threshold Points..... 48
Appendix C – Complete R Code..... 49

1. Introduction

The goal of this project is to build and compare several classification models to **predict which customers will subscribe to a term deposit** as part of a bank's telemarketing campaign.

The classification problem is challenging because:

- The dataset is **highly imbalanced**: only about 11–12% of customers say *"yes"* to the term deposit.
- There are many **categorical variables with many levels** (job, education, month, etc.).
- Some variables are **strongly related to the outcome** in a way that can easily leak information into the model (most notably `duration`).

Our objectives were:

1. Carefully **preprocess** the data and engineer meaningful features, especially for contact history (`pdays`).
2. Fit both **linear** and **non-linear** models (logistic regression, penalized logistic, LDA, PLSDA, KNN, neural network, FDA).
3. Use **repeated cross-validation** and **ROC/AUC** as our main metric to compare models in a way that is robust to class imbalance.
4. Evaluate all models on a **held-out test set**, explore **probability thresholds**, and justify a **final model choice** from both performance and interpretability perspectives.

---

2. Data and Initial Cleaning

2.1 Dataset Source and Description

The data come from the **Bank Marketing** dataset in the UCI Machine Learning Repository. It contains information from **direct marketing campaigns (phone calls)** conducted by a Portuguese banking institution. The target variable `y` indicates whether a client subscribed a term deposit ("yes"/"no"). (UCI Machine Learning Repository)

We specifically used the **`bank-additional-full.csv`** file:

- **Number of observations:** 41,188
- **Number of input variables:** 20, plus the binary response `y`
- **Time period:** From **May 2008 to November 2010**, ordered by date in the file (UCI Machine Learning Repository)
- **Problem type:** Binary classification (`yes` vs `no`)

The predictors cover:

- **Client attributes:** age, job, marital status, education, default, housing loan, personal loan
- **Campaign/contact info:** contact type, last contact month/day, number of contacts in this campaign (`campaign`), number of days since last contact (`pdays`), number of previous contacts (`previous`), outcome of previous campaigns (`poutcome`)
- **Macro-economic indicators:** employment variation rate (`emp.var.rate`), consumer price index, consumer confidence index, Euribor 3-month rate, number of employees (`nr.employed`)

We converted the response variable `y` into a factor with levels `("no","yes")`, treating `"yes"` as the positive class.

2.2 Loading the Data and First Look

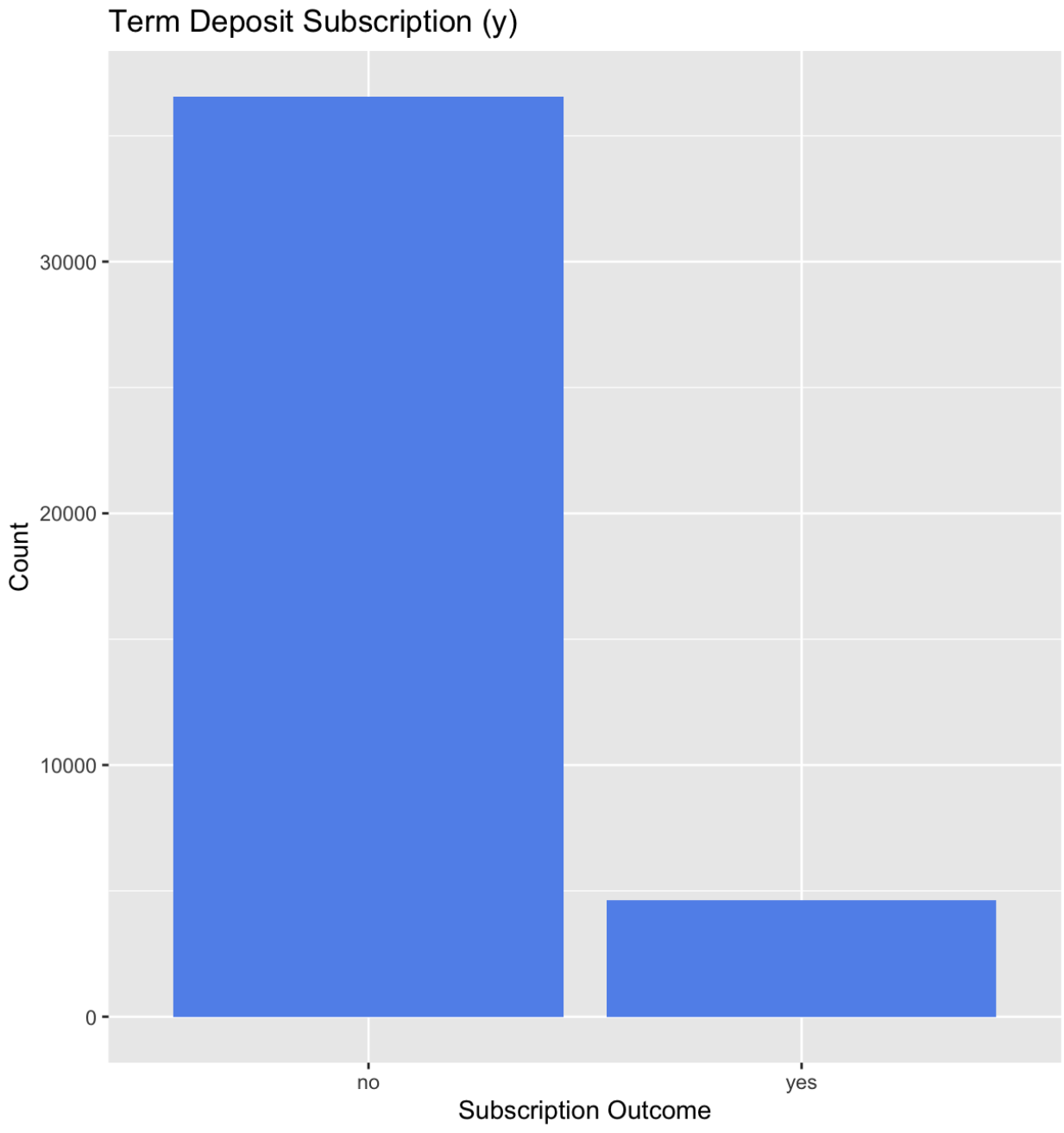
We loaded the CSV with `sep=";"` and checked:

- basic structure (`str()`),
- summary statistics, and
- the distribution of the outcome `y`.

Outcome prevalence:

- Overall: **~11.3% “yes”**, **88.7% “no”**.

This immediately signaled **class imbalance**, which drove later choices (SMOTE, ROC/AUC, threshold tuning).



2.3 Removing the Leakage Variable `duration`

From the UCI description and from exploratory checks, we noticed that the variable **`duration`** (last contact duration in seconds) is almost deterministically related to `y`:

- Long successful calls tend to be *very* predictive of "yes".
- However, **`duration` is only known *after* the call ends**—not at the time when we decide *whom* to call.

So:

- At first glance we saw that customers with **very short durations** almost never subscribed, and long-duration calls had **very high subscription rates**.
- This is a classic example of **data leakage**: it uses information from *after* the decision (call length) to predict the outcome.

Because the project is about **targeting** customers *before* calling them, we dropped `duration` immediately:

> **Design decision:**
>
> - **Drop `duration` at the start** to avoid inflated performance and to mimic a realistic *pre-call* decision scenario.
> - This follows the recommendation in the dataset's original paper and many existing analyses.

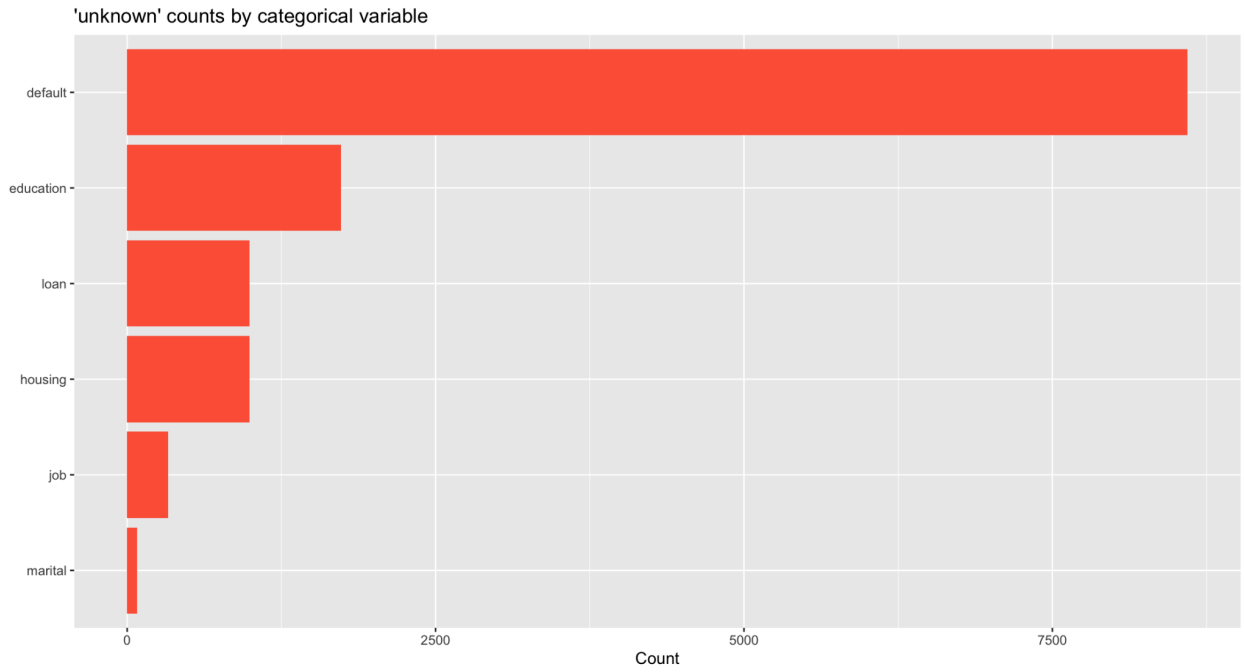
```
duration_summary
A tibble: 2 × 5
y         n mean_duration median_duration sd_duration
<fct> <int>         <dbl>           <dbl>       <dbl>
no    36548           221.            164.        207.
yes    4640           553.            449         401.
```

2.4 Handling Missing Values and `"unknown"` Levels

The dataset does not use `NA` extensively but instead uses `"unknown"` for many categorical variables (`job`, `education`, `marital`, `default`, `housing`, `loan`, etc.).

We:

1. **Counted real NAs** for each variable (almost none).
2. Counted `"unknown"` levels for all categorical predictors and plotted them.



Design decision:

- We **did not impute** `"unknown"`; instead we **treated it as its own category** (factor level).
- Justification:
    - In marketing, “unknown” often carries information (e.g., missing credit info may correlate with higher risk or lower engagement).
    - Imputing arbitrary values could **wash away** that signal.

---

3. Feature Engineering and Preprocessing

3.1 Special Handling of `pdays`

The variable `pdays` measures the **number of days since the client was last contacted in a previous campaign**, with **`999` meaning "never contacted before"**.

At first glance, `pdays` is:

- highly skewed, and
- overloaded with meaning: a number *and* a sentinel code.

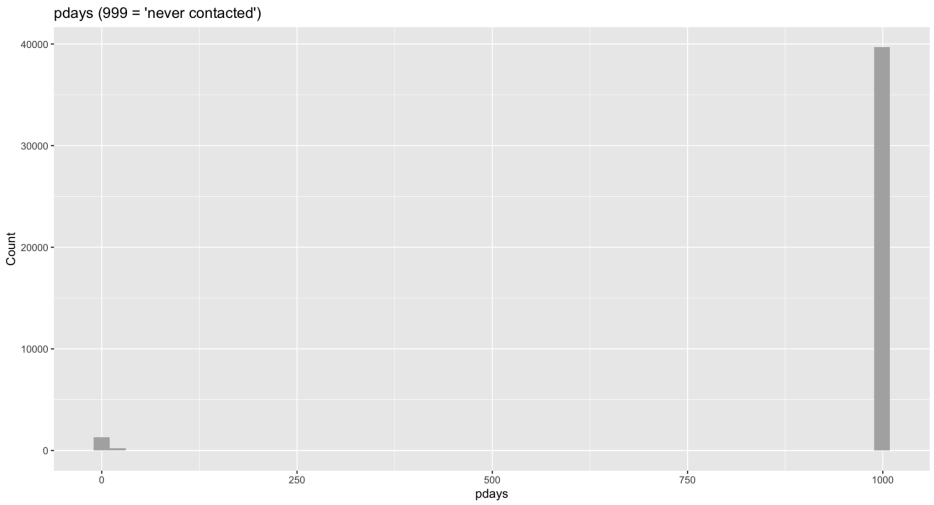
We decided this needed special treatment rather than being used raw.

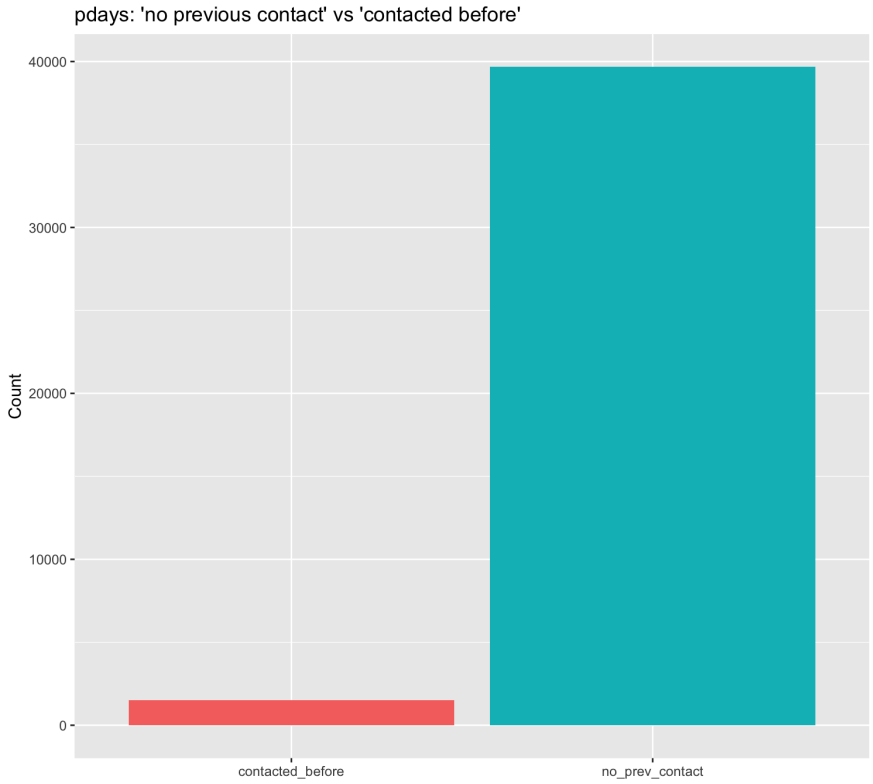
We engineered **three interpretable features**:

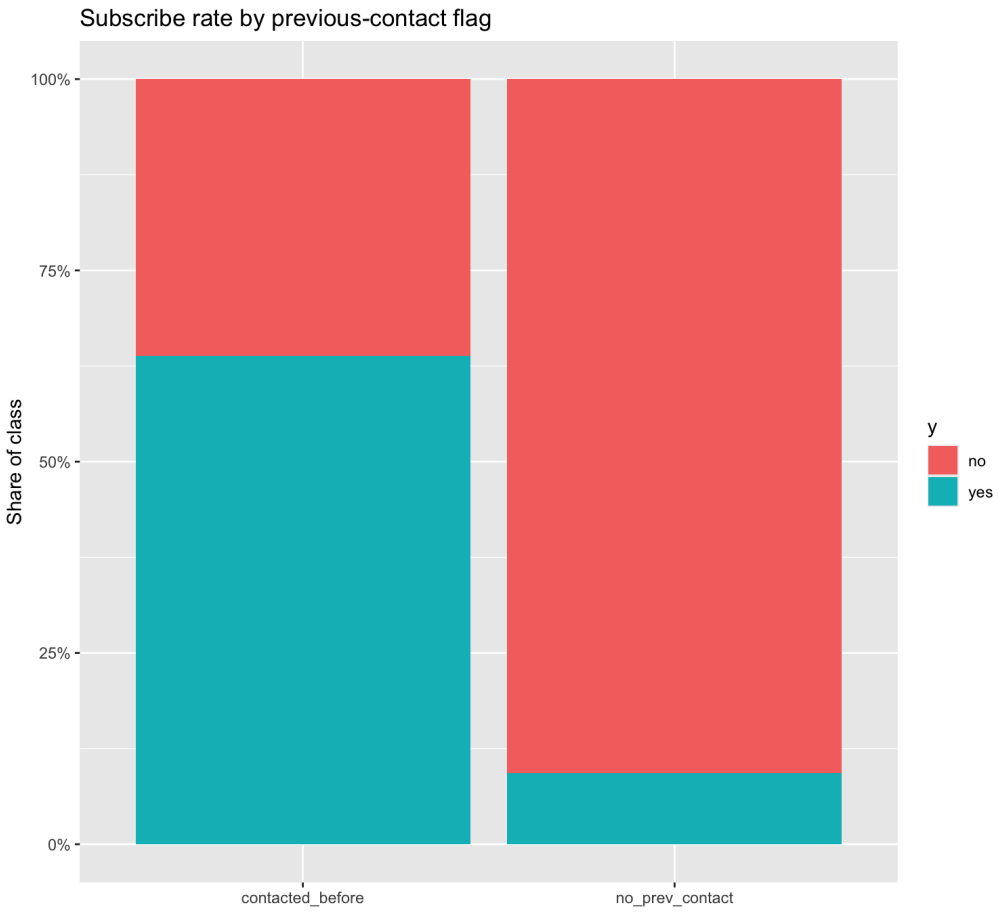
1. **`contacted_before` (yes/no factor)**
    - `contacted_before = "yes"` if `pdays < 999`, otherwise `"no"`.
    - Captures whether the client **ever** had a prior contact.
2. **`pdays_bucket` (categorical recency group)**
    - For clients with previous contact, we binned `pdays` into:
        - `0–7`, `8–30`, `31–90`, `91–180`, `181–365`, `>365`
    - For those with `pdays = 999`, we set `pdays_bucket = "no_prev_contact"`.
    - This allows **non-linear effects** of recency (e.g., very recent vs very old contact).
3. **`pdays_recency` (continuous recency score)**
    - Defined as `1 / (pdays + 1)` for `pdays < 999`, and `0` for `pdays == 999`.
    - This monotonically increasing score gives **higher values to more recent contacts**, but compresses large `pdays`.

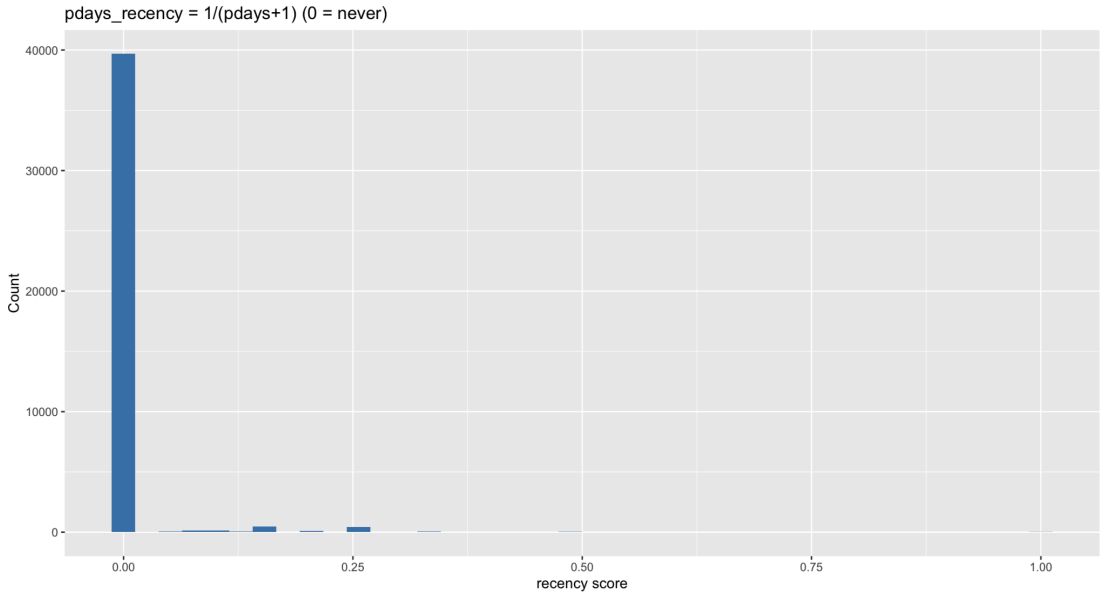
We then **dropped the original `pdays`** column to avoid redundancy.

Figure placeholders:









These plots showed that:

- Clients with **previous successful contact** and **more recent contact** had **higher subscription rates**, confirming that this recency information is valuable.

3.2 Numeric Variables: Skewness, Transforms, and Correlation

We identified all numeric predictors (after `pdays` engineering) and computed **skewness** and histograms.

The skewness table (before transformation) showed:

- **Highly right-skewed:** `pdays_recency`, `campaign`, `previous`
- **Moderately left-skewed:** `nr.employed`
- Others (`age`, `emp.var.rate`, `euribor3m`, `cons.conf.idx`, `cons.price.idx`) were moderately skewed or near symmetric.

Figure placeholder:

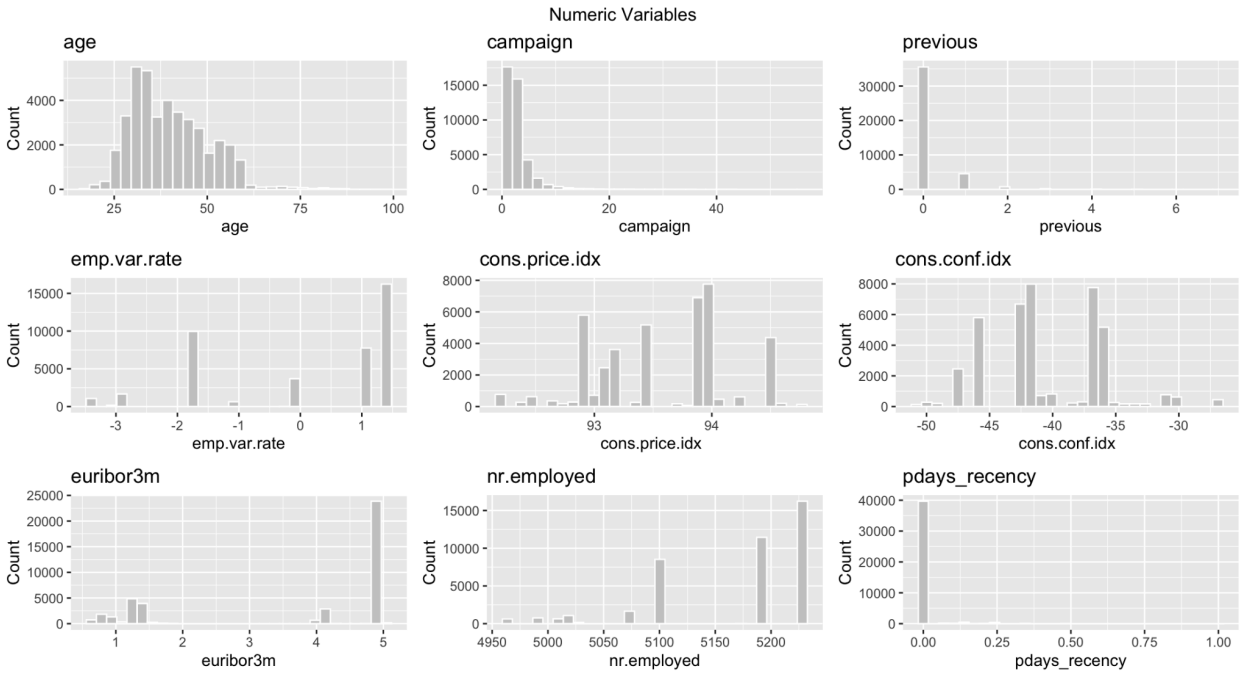


Figure 8: Grid of histograms for all numeric variables (before transform).

```
print(skew_tbl_before)
A tibble: 9 × 3
variable       skewness skew_class
<chr>             <dbl> <chr>
pdays_recency      9.60  high right-skew
campaign           4.76  high right-skew
previous           3.83  high right-skew
nr.employed       -1.04  high left-skew
age                0.785 near-symmetric/moderate
emp.var.rate      -0.724 near-symmetric/moderate
euribor3m         -0.709 near-symmetric/moderate
cons.conf.idx      0.303 near-symmetric/moderate
cons.price.idx    -0.231 near-symmetric/moderate
```

Figure 9: Table of skewness .

Transform decisions:

- For **right-skewed count variables** `campaign` and `previous`, we used **Box–Cox transforms** (after adding +1 to avoid zeros).
    - Rationale: counts with long right tails benefit from Box–Cox to compress large values and stabilize variance.
- For **left-skewed or possibly non-positive variables**, we allowed **Yeo–Johnson**, but we explicitly **excluded `pdays_recency`** from both transform sets:
    - `pdays_recency` is already a **nonlinear transform** (1/(pdays+1)) with a clear interpretation.
    - Applying another power transform on top of that would reduce interpretability without clear gain.

We then plotted the **post-transform histograms** to confirm visually that:

- Distributions became more symmetric
- Extreme values were compressed appropriately

Figure placeholder:

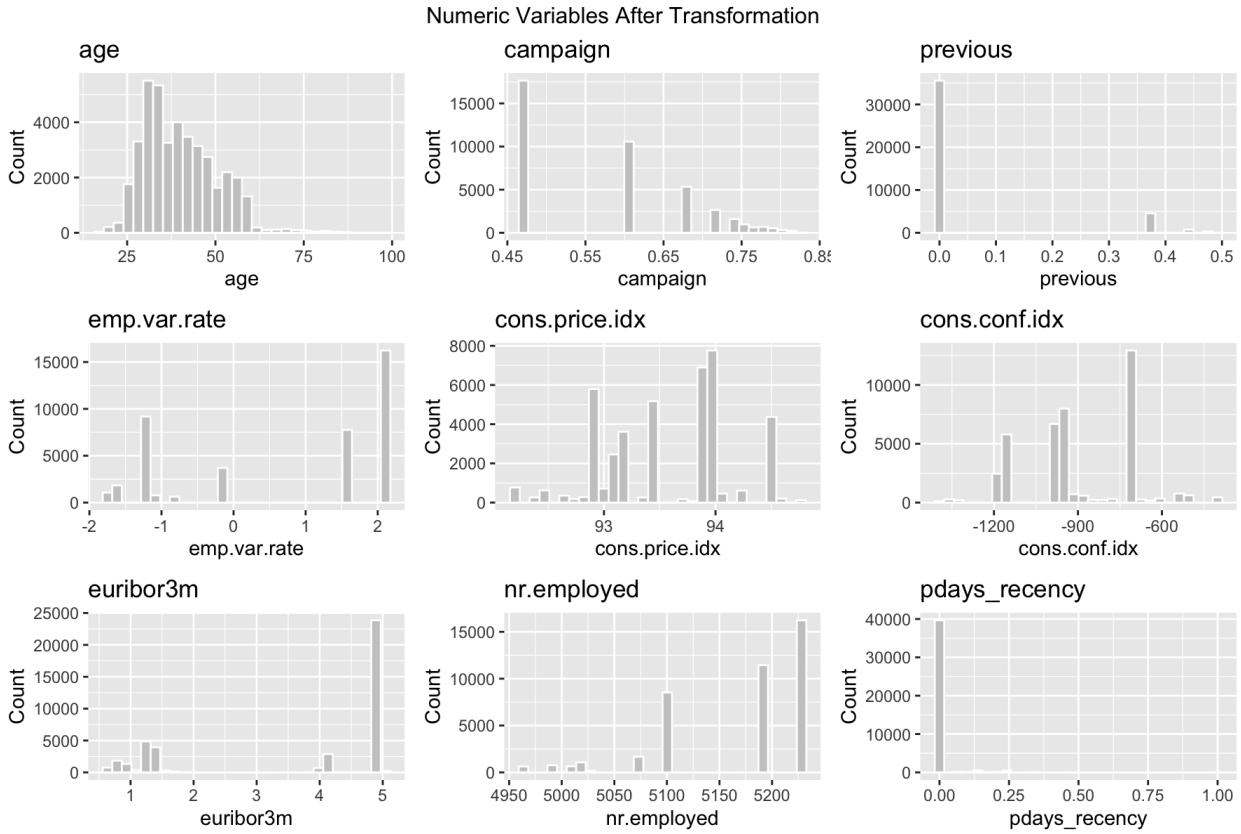


Figure 10: Grid of histograms for numeric variables after transformation.

Numeric NZV filtering

We computed **near-zero variance (NZV)** metrics for numeric variables and found:

- `pdays_recency` behaved as NZV (almost all zeros with a few small positive values).

Given that we already had **`contacted_before`** **and** **`pdays_bucket`** representing similar information more transparently, we decided to **drop** **`pdays_recency`** from the numeric block.

```
print(nzv_tbl_num)
     variable freqRatio percentUnique zeroVar  nzv
pdays_recency   90.3713    0.06555307   FALSE TRUE
```

Figure 11: NZV numeric table (showing `pdays_recency` as NZV).

Numeric correlation filtering

We then computed a correlation matrix for the transformed numeric variables and used a **0.90 cutoff** to remove highly correlated predictors.

We dropped:

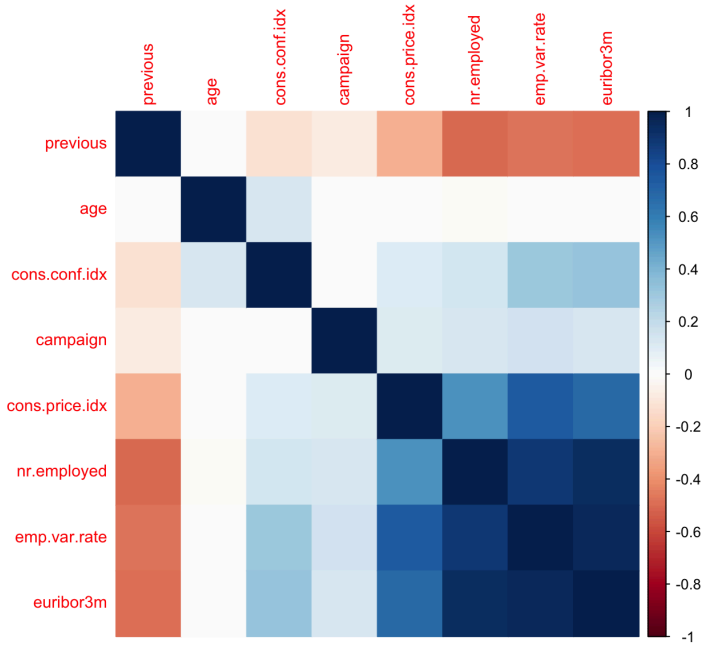
- `emp.var.rate`
- `euribor3m`

These macro-economic indicators are known to be highly correlated in this dataset (e.g., employment variation rate vs Euribor/interest rates). (UCI Machine Learning Repository)

Design reasoning:

- Strong collinearity can make linear models unstable and inflate variance.
- Since our goal is **prediction** rather than interpret each macro-economic effect separately, we allow the model to keep a subset of them and drop the rest.

Figure placeholders:



- *Figure 12: Correlation heatmap before dropping high-correlation numerics.*

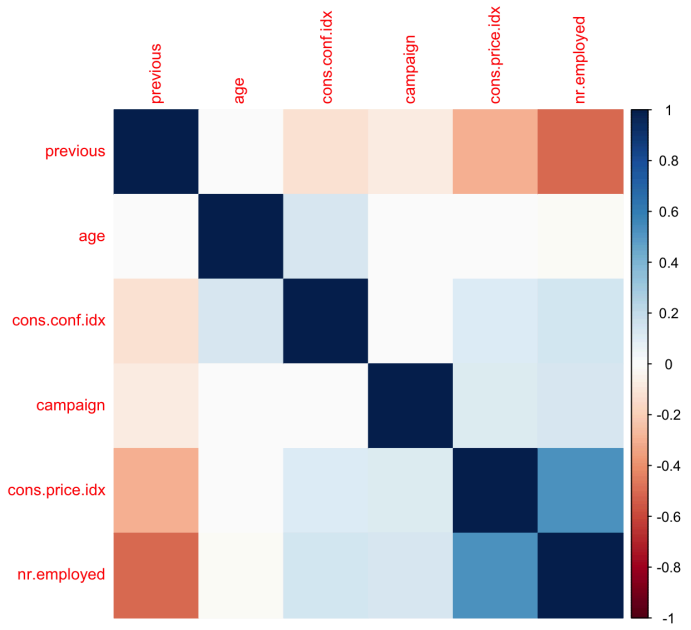


Figure 13: Correlation heatmap after dropping `emp.var.rate` and `euribor3m`.

3.3 Outlier Handling and Spatial Sign + Center/Scale

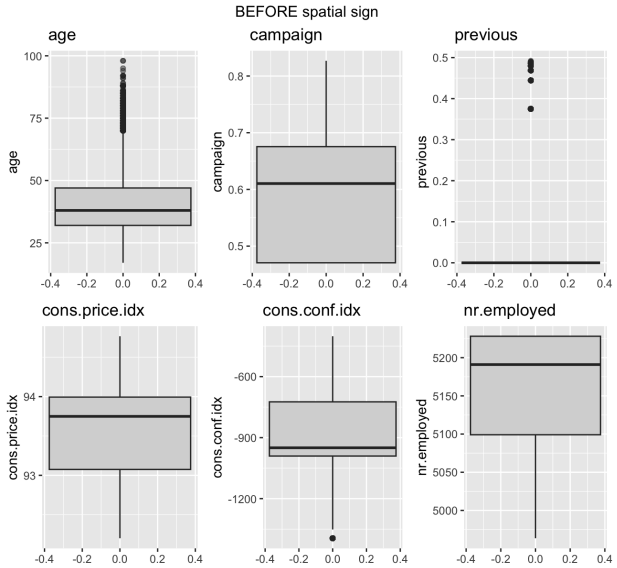
Before scaling, we used boxplots to inspect numeric **outliers**.

We then applied:

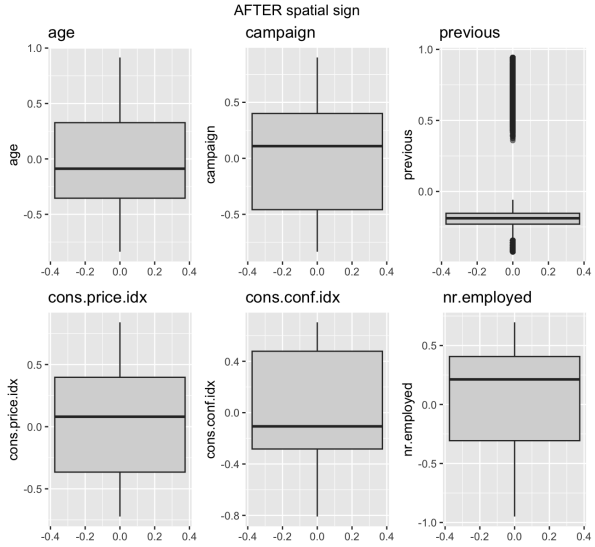
1. **Spatial Sign transformation** on the numeric block.
    - This maps each numeric observation to the **unit sphere**, emphasizing **direction** of the numeric vector rather than magnitude.
    - Purpose:
        - Reduce the influence of **extreme outliers**.
        - Make distance-based methods (like KNN) more stable.
        - Follow the specific preprocessing order recommended in class notes.
2. **Centering and scaling** each numeric variable (mean 0, variance 1).

- Needed for:
  - Algorithms sensitive to scale (KNN, neural nets).
  - Penalized methods (GLMNet).
  - Distance in MARS/FDA basis functions.

Figure placeholders:



- *Figure 14: Numeric boxplots before spatial sign.*



- *Figure 15: Numeric boxplots after spatial sign.*

3.4 Categorical Variables and Dummy Encoding

All remaining non-numeric predictors (excluding `y`) were converted to **factors**.

We then used **dummy (one-hot) encoding** on the **entire dataset** (per project instructions), with `fullRank = FALSE` to get a full set of indicator columns for each level.

- Before dummies: **24 predictors**
- After dummies: **74 columns**

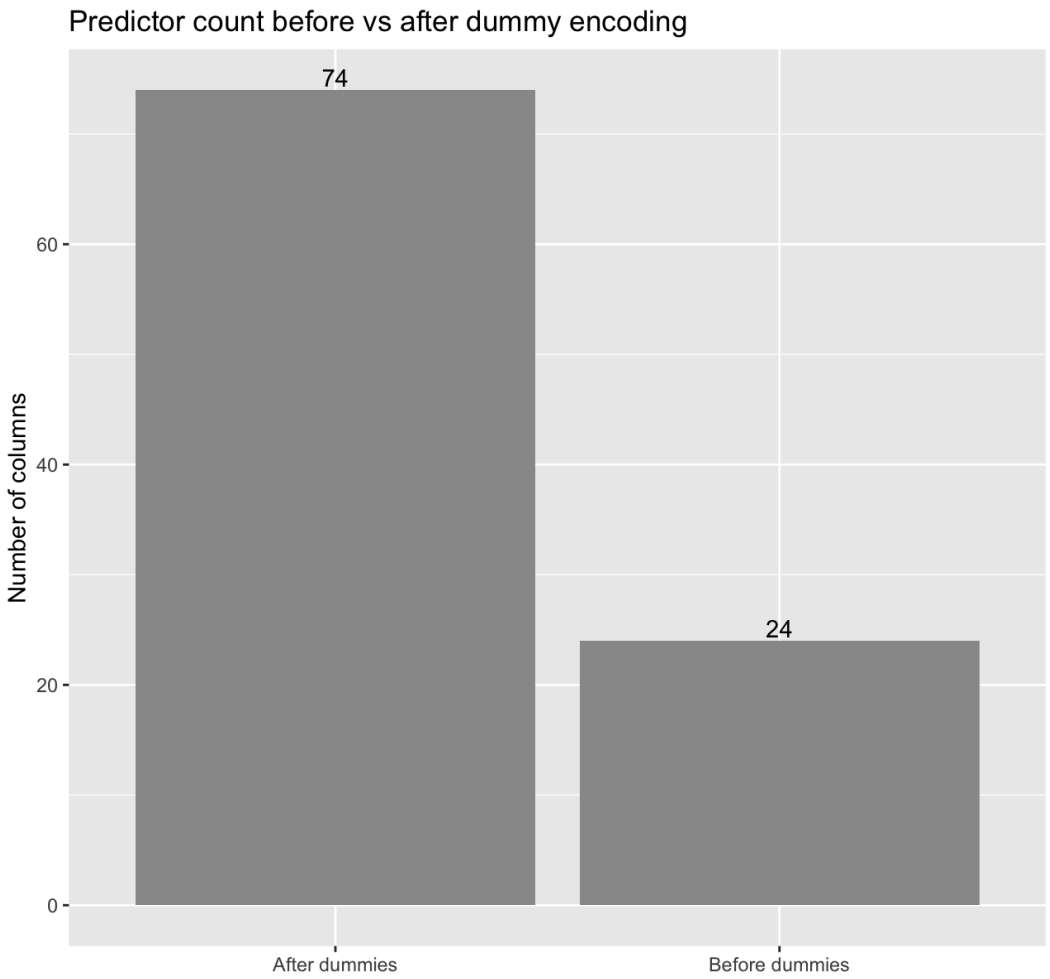


Figure 16: Bar plot showing predictor count "Before dummies: 24" vs "After dummies: 74".

Reasoning:

- Most modeling functions in `caret` expect numeric input; dummy encoding allows us to pass **all models the same design matrix**.
- We chose **full rank = FALSE** (no reference level dropped at this stage) because:
    - Later steps and algorithms (like GLMNet) handle linear dependence internally.
    - It simplifies interpretation (each level has its own column).

3.5 NZV and Correlation Filtering on Dummies

With 74 dummy columns, some represented extremely rare levels or redundant codings.

We:

1. Ran **NZV detection** on dummy columns and removed variables with:
    - very low variance,
    - extremely unbalanced frequency (e.g., very rare job or month), or
    - zero variance.

Examples removed:

- `education.illiterate`, `marital.unknown`, `job.unknown`, some rare months (`month.dec`, `month.sep`, `month.oct`, etc.),
- Rare `pdays_bucket` levels that were essentially never observed.

These were **rare patterns** that would add noise, make modeling harder, and bring almost no predictive information.

```
> print(nzv_tbl)
                       variable   freqRatio percentUnique zeroVar  nzv
1                   default.yes 13728.33333   0.004855783   FALSE TRUE
2           education.illiterate  2287.22222   0.004855783   FALSE TRUE
3                marital.unknown   513.85000   0.004855783   FALSE TRUE
4                      month.dec   225.30769   0.004855783   FALSE TRUE
5                    job.unknown   123.81212   0.004855783   FALSE TRUE
6              pdays_bucket.8-30   120.85799   0.004855783   FALSE TRUE
7                      month.mar    74.43590   0.004855783   FALSE TRUE
8                      month.sep    71.25965   0.004855783   FALSE TRUE
9                      month.oct    56.36490   0.004855783   FALSE TRUE
10                   job.student    46.07200   0.004855783   FALSE TRUE
11               housing.unknown    40.60404   0.004855783   FALSE TRUE
12                  loan.unknown    40.60404   0.004855783   FALSE TRUE
13                job.unemployed    39.61933   0.004855783   FALSE TRUE
14                 job.housemaid    37.85660   0.004855783   FALSE TRUE
15              pdays_bucket.0-7    33.99405   0.004855783   FALSE TRUE
16              poutcome.success    28.99854   0.004855783   FALSE TRUE
17             job.self-employed    27.98522   0.004855783   FALSE TRUE
18              job.entrepreneur    27.28846   0.004855783   FALSE TRUE
19           contacted_before.no    26.18680   0.004855783   FALSE TRUE
20          contacted_before.yes    26.18680   0.004855783   FALSE TRUE
21 pdays_bucket.no_prev_contact    26.18680   0.004855783   FALSE TRUE
22                   job.retired    22.94651   0.004855783   FALSE TRUE
23             education.unknown    22.79434   0.004855783   FALSE TRUE
24            pdays_bucket.31-90     0.00000   0.002427892    TRUE TRUE
25           pdays_bucket.91-180     0.00000   0.002427892    TRUE TRUE
26          pdays_bucket.181-365     0.00000   0.002427892    TRUE TRUE
27             pdays_bucket.>365     0.00000   0.002427892    TRUE TRUE
```

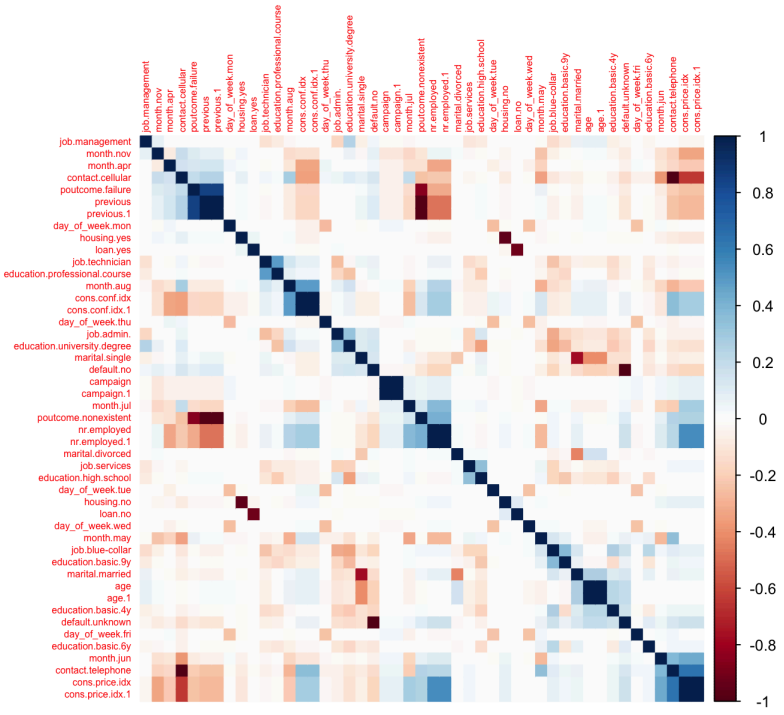
Figure 17: NZV dummy table screenshot.

2. Ran **correlation filtering** (again 0.90 cutoff) on the dummy matrix.

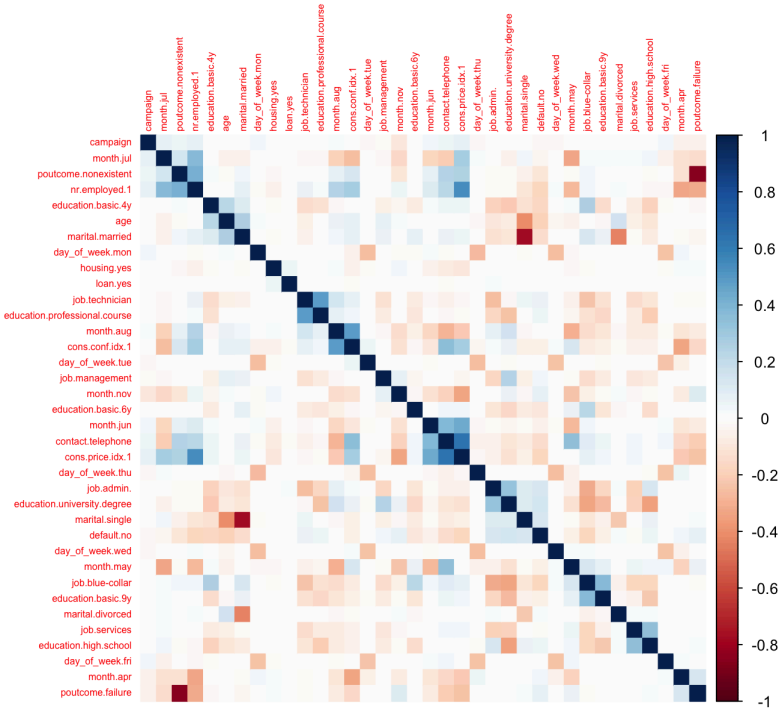
Dropped, for example:

- `cons.price.idx`, `cons.conf.idx`, `nr.employed` dummy variants (strongly tied to economic conditions),
- Some **duplicated or nearly linear dependent dummies** (e.g., `previous` vs `previous.1`, `housing.no`, `loan.no`).

This step reduces multicollinearity and improves numerical stability for models like logistic regression and LDA.



- *Figure 18: Dummy correlation heatmap before dropping.*



- *Figure 19: Dummy correlation heatmap after dropping.*

3.6 Final Design Matrix and Predictor Count

After all numeric + categorical preprocessing and filtering, we constructed:

- `final_df = data.frame(Xdum_filtered, y = y_all)`

When fitting models with `caret`, the output consistently reports:

- **Samples:** 32,951 in training
- **Predictors: 36**

So although dummy encoding temporarily produced 70+ columns, after NZV and correlation cleaning, plus the numeric block, the **effective number of predictors used by the models is 36**.

We deliberately **completed all preprocessing before splitting** into train/test (following project instructions), but we are aware that in strict practice many steps (e.g., NZV, correlations, transforms) would normally be fit on training only and then applied to test. Here we followed the **course's required workflow**.

---

4. Train–Test Split and Class Imbalance

4.1 Stratified Train–Test Split

We performed a **stratified 80/20 split** on the final dataset:

- Training set: **32,951 observations**
- Test set: **8,237 observations**
- Stratification on `y` ensured both sets preserve the original class proportions.

Outcome proportions:

- Training: `no` ~ 88.73%, `yes` ~ 11.27%
- Test: `no` ~ 88.73%, `yes` ~ 11.27%

```
> prop.table(table(train_df$y))

       no       yes
0.8873479 0.1126521
> prop.table(table(test_df$y))

       no       yes
0.8873376 0.1126624
```

Figure 20: Console screenshot of `prop.table(table(train_df$y))` and `prop.table(table(test_df$y))`.

4.2 Class Imbalance and Implications

Because only ~11% of customers subscribe:

- A naive classifier that always predicts **"no"** would already achieve **~88.7% accuracy**.
- Therefore, **accuracy alone** is misleading.

This motivated:

- the use of **ROC/AUC** and **balanced accuracy** instead of raw accuracy,
- **SMOTE** during training to better learn the minority "yes" class,
- and explicit **threshold tuning** later to trade off sensitivity vs specificity.

---

5. Resampling Strategy, SMOTE, and Evaluation Metric

5.1 Repeated Cross-Validation Setup

For all models, we used `caret::train()` with a **consistent resampling strategy**:

- **10-fold cross-validation**
- **3 repeats** (total of 30 resamples)
- `classProbs = TRUE` to obtain predicted probabilities

- `summaryFunction = twoClassSummary` to compute:
    - ROC (AUC),
    - sensitivity,
    - specificity

This gives stable estimates of each model's discriminative ability.

5.2 Why ROC/AUC (and not just Accuracy or Kappa)?

ROC/AUC was chosen as the **primary tuning metric** (`metric = "ROC"`) for several reasons:

1. **Threshold-independence**
    - ROC/AUC evaluates performance over **all possible probability cutoffs**, not just 0.5.
    - This is crucial when we later change thresholds based on marketing costs/benefits.
2. **Robustness to class imbalance**
    - AUC is not dominated by the majority class in the same way that accuracy is.
    - With ~11% positives, maximizing accuracy would mostly optimize how we classify "no".
3. **Direct comparison across models**
    - ROC/AUC provides a **single scalar** (0.5 = random, 1 = perfect) that compares linear vs non-linear models fairly.

We still recorded:

- **Accuracy** and **Kappa** (on the test set),
- **Sensitivity**, **Specificity**, **Balanced Accuracy**, **PPV**, and **NPV**.

But for **model tuning and selection**, ROC/AUC was more aligned with our problem and the class imbalance. Kappa, while valuable, is still more threshold-dependent and less directly interpretable than AUC for this project.

5.3 Why SMOTE, and Where It Was Used

To address class imbalance during training, we used **SMOTE (Synthetic Minority Over-sampling Technique)** via `sampling = "smote"` in `trainControl` (using the `themis` recipes backend).

- SMOTE goes beyond simple oversampling of the minority class by **creating synthetic "yes" examples** in feature space.
- This helps the model see **more varied minority patterns** without simply duplicating existing cases.

Because `caret` applies SMOTE **inside each CV fold**, it avoids leaking information from test folds into training folds.

We used **two trainControl objects**:

1. `ctrl` (with SMOTE): used for
    - Logistic regression (GLM)
    - Flexible Discriminant Analysis (FDA)
    - Neural Network (nnet)
2. `ctrl_nosmote` (no SMOTE): used where SMOTE caused technical issues:
    - GLMNet (penalized logistic)
    - PLSDA
    - LDA
    - KNN

For those models, we still relied on **class weights implicitly via ROC metric** and subsequent threshold tuning, but without SMOTE.

We also briefly considered alternatives such as:

- **Random undersampling** of “no”,
- **ROSE** or other synthetic methods,
- **Cost-sensitive learning** (loss weights).

Given the class notes and built-in support for SMOTE with `themis`, we chose SMOTE as the primary imbalance strategy.

---

6. Models Fit and Tuning Decisions

We fit the following models using the same preprocessed predictors and consistent resampling:

- **Linear models**
    - Logistic regression (GLM)
    - Penalized logistic regression (GLMNet)
    - LDA
    - PLSDA
- **Non-linear models**
    - Neural Network (nnet)
    - Flexible Discriminant Analysis (FDA)
    - k-Nearest Neighbors (KNN)
    - (SVM radial was attempted but dropped due to convergence issues.)

For each model we tuned hyperparameters to **maximize ROC**.

6.1 Baseline Logistic Regression (GLM)

- Method: `method = "glm"`, `family = binomial`
- Control: `ctrl` with SMOTE and ROC metric

Cross-validated performance (mean over 30 resamples):

- **ROC:** 0.7905
- **Sensitivity:** ~0.846
- **Specificity:** ~0.636

Interpretation:

- Logistic regression, even with **linear decision boundary in the transformed feature space**, performs **very strongly**, with good discrimination between “yes” and “no”.
- SMOTE increased attention to the minority class, leading to high sensitivity.

We treat this as the **main linear baseline**.

```
> print(mod_glm)
Generalized Linear Model

32951 samples
   36 predictor
    2 classes: 'no', 'yes'

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 3 times)
Summary of sample sizes: 29656, 29656, 29656, 29655, 29655, 29656, ...
Addtional sampling using SMOTE

Resampling results:

  ROC        Sens       Spec
  0.7904895  0.8460047  0.6358706
```

Figure 21: Console printout of `mod_glm` summary (showing ROC, Sens, Spec).

6.2 Penalized Logistic Regression (GLMNet)

- Method: `method = "glmnet"`
- Control: `ctrl_nosmote` (no SMOTE, due to issues when SMOTE was used)
- Tuning grid:
    - **alpha:** {0 (ridge), 0.5 (elastic net), 1 (lasso)}
    - **lambda:** 10^seq(-4, -1, length = 7), from 1e-4 to 1e-1

Design reasoning:

- Penalization can:
    - Reduce overfitting.
    - Perform variable selection (lasso).
    - Handle correlated predictors more gracefully.

Cross-validated ROC:

- Very similar to baseline logistic (around **0.7901** at best).
- Optimal tuning chosen by caret:
    - **alpha = 1** (pure lasso)
    - **lambda = 1e-4**

This indicates only **mild regularization**; the data and preprocessing were stable enough that we didn't need heavy penalization.

```
Resampling results across tuning parameters:

  alpha  lambda        ROC        Sens       Spec
  0.0    0.0001000000  0.7883615  0.9897397  0.1805894
  0.0    0.0003162278  0.7883615  0.9897397  0.1805894
  0.0    0.0010000000  0.7883615  0.9897397  0.1805894
  0.0    0.0031622777  0.7883615  0.9897397  0.1805894
  0.0    0.0100000000  0.7883074  0.9898765  0.1795117
  0.0    0.0316227766  0.7859128  0.9959073  0.1026418
  0.0    0.1000000000  0.7816291  1.0000000  0.0000000
  0.5    0.0001000000  0.7900786  0.9856926  0.2184841
  0.5    0.0003162278  0.7900849  0.9857268  0.2184844
  0.5    0.0010000000  0.7899719  0.9866046  0.2121089
  0.5    0.0031622777  0.7888019  0.9880867  0.1983697
  0.5    0.0100000000  0.7851938  0.9921908  0.1519428
  0.5    0.0316227766  0.7762026  1.0000000  0.0000000
  0.5    0.1000000000  0.7364043  1.0000000  0.0000000
  1.0    0.0001000000  0.7901071  0.9857382  0.2184844
  1.0    0.0003162278  0.7901018  0.9860232  0.2165990
  1.0    0.0010000000  0.7897383  0.9870835  0.2095948
  1.0    0.0031622777  0.7871181  0.9888847  0.1854392
  1.0    0.0100000000  0.7812366  0.9952119  0.1055174
  1.0    0.0316227766  0.7606474  1.0000000  0.0000000
  1.0    0.1000000000  0.5000000  1.0000000  0.0000000

ROC was used to select the optimal model using the largest value.
The final values used for the model were alpha = 1 and lambda = 1e-04.
```

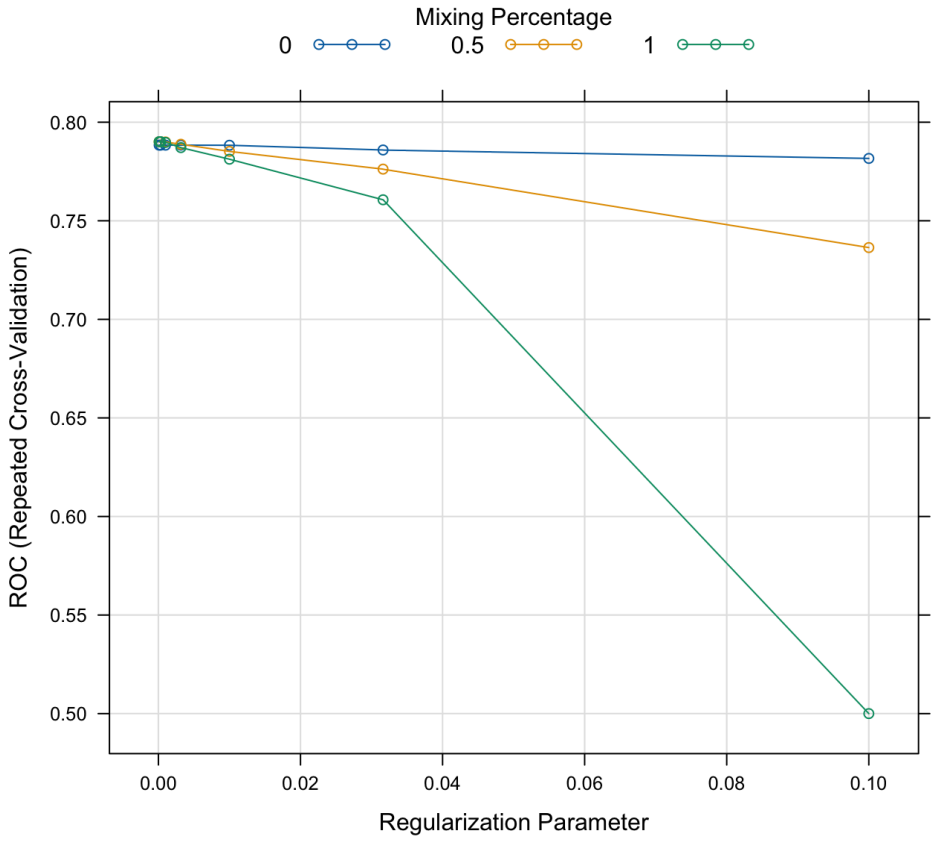


Figure 22: Console print for `mod_glmnet` with ROC across (alpha, lambda).

6.3 Flexible Discriminant Analysis (FDA)

- Method: `method = "fda"` (with MARS basis)
- Control: `ctrl` (with SMOTE)
- Tuning:
    - `tuneLength = 10` let caret pick about 10 values of `nprune` (number of basis functions); `degree` fixed at 1.

Cross-validated best model:

- **nprune = 5**, **degree = 1**
- ROC ≈ **0.7747**
- Sensitivity ≈ 0.801
- Specificity ≈ 0.655

Interpretation:

- FDA represents a **non-linear extension of LDA** via piecewise linear basis functions.
- It improved ROC slightly over a naive linear boundary but **did not surpass** the logistic/GLMNet performance.
- SMOTE helps FDA give more balanced decisions between “yes” and “no”.

```
Resampling results across tuning parameters:

  nprune  ROC        Sens       Spec
   2      0.7451342  0.7194157  0.7190206
   3      0.7517379  0.7368923  0.7061804
   4      0.7517178  0.7376447  0.7056418
   5      0.7746625  0.8009849  0.6551766
   6      0.7729353  0.8142544  0.6416181
   8      0.7738271  0.8205706  0.6372178
   9      0.7736915  0.8381955  0.6206919
  10      0.7739723  0.8382298  0.6246459
  11      0.7739691  0.8474867  0.6142297
  13      0.7742514  0.8572569  0.6055193

Tuning parameter 'degree' was held constant at a value of 1
ROC was used to select the optimal model using the largest value.
The final values used for the model were degree = 1 and nprune = 5.
```

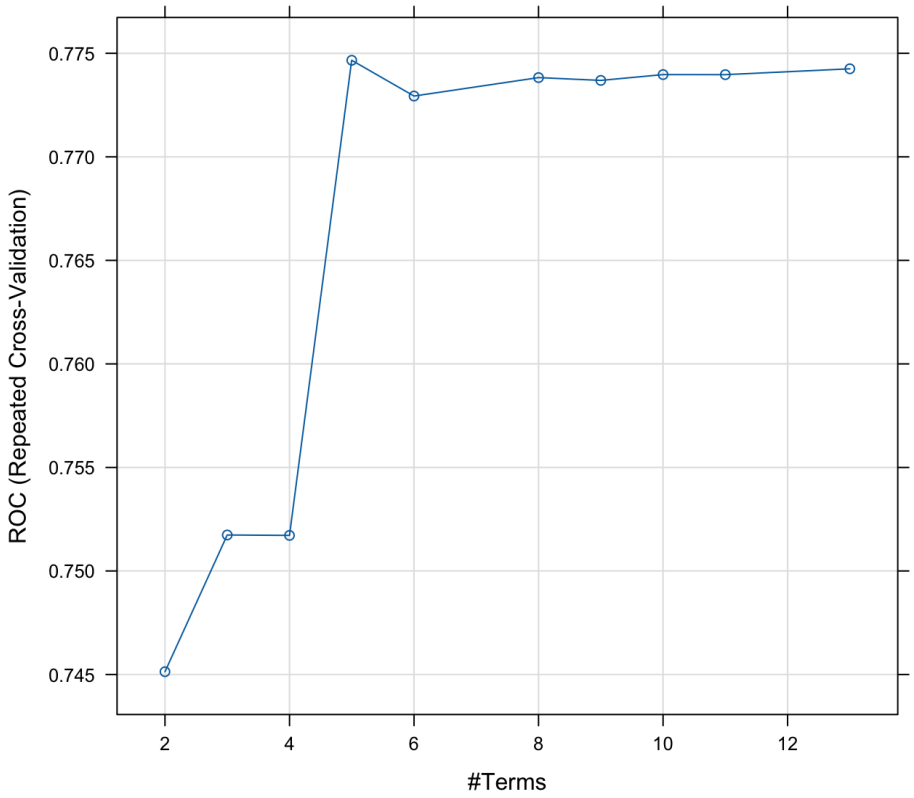


Figure 24: Plot of ROC vs `nprune` (output of `plot(mod_fda)`).

6.4 Neural Network (Single Hidden Layer)

- Method: `method = "nnet"`
- Control: `ctrl` (with SMOTE)
- Tuning grid:
    - **size:** {3, 5, 7} (number of hidden units)
    - **decay:** {0.001, 0.01, 0.1} (weight decay regularization)

Design reasoning:

- A single hidden layer MLP can capture **non-linear interactions** between predictors.

- We kept the network relatively small to avoid overfitting given the dataset size and complexity.

- Larger networks with more hidden nodes would require more careful tuning and computational cost.

Cross-validated best model:

- **size = 3**, **decay = 0.1**

- ROC ≈ **0.7886**

- Sensitivity ≈ 0.868

- Specificity ≈ 0.617

This model is **slightly below** logistic/GLMNet on CV ROC, but still very competitive.

```
Resampling results across tuning parameters:

  size  decay  ROC        Sens       Spec
  3     0.001  0.7873204  0.8602442  0.6215049
  3     0.010  0.7870101  0.8656473  0.6136058
  3     0.100  0.7885730  0.8677454  0.6166577
  5     0.001  0.7776660  0.8612698  0.6064212
  5     0.010  0.7828713  0.8614062  0.6168273
  5     0.100  0.7854679  0.8627632  0.6126991
  7     0.001  0.7742325  0.8525026  0.6113536
  7     0.010  0.7796614  0.8514428  0.6165575
  7     0.100  0.7810437  0.8628888  0.6060516

ROC was used to select the optimal model using the largest value.
The final values used for the model were size = 3 and decay = 0.1.
```

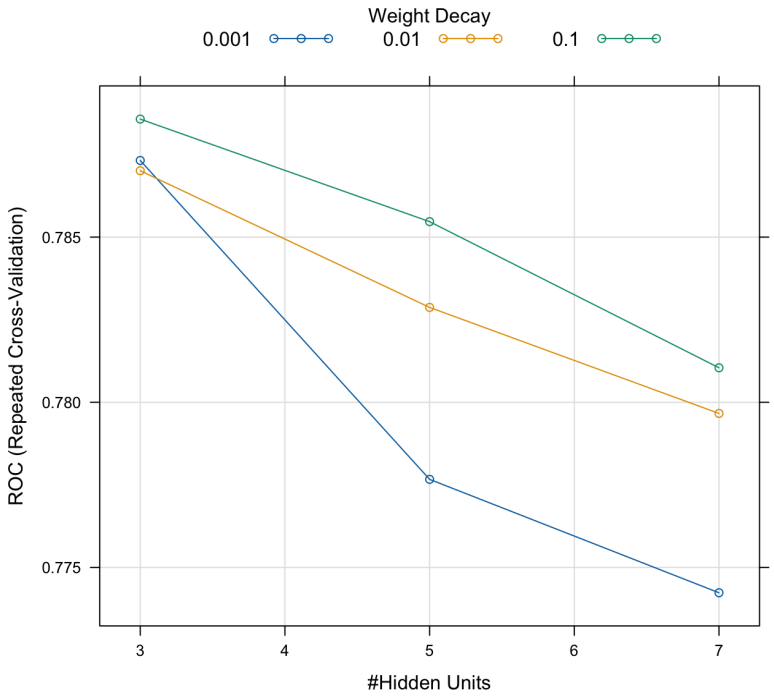


Figure 25: `plot(mod_nnet)` showing ROC vs (size, decay).

6.5 Partial Least Squares Discriminant Analysis (PLSDA)

- Method: `method = "pls"` (when outcome is factor, caret performs PLSDA)
- Control: `ctrl_nosmote` (SMOTE caused dimension errors)
- Tuning:
    - `tuneLength = 15` → up to 15 latent components (`ncomp`)

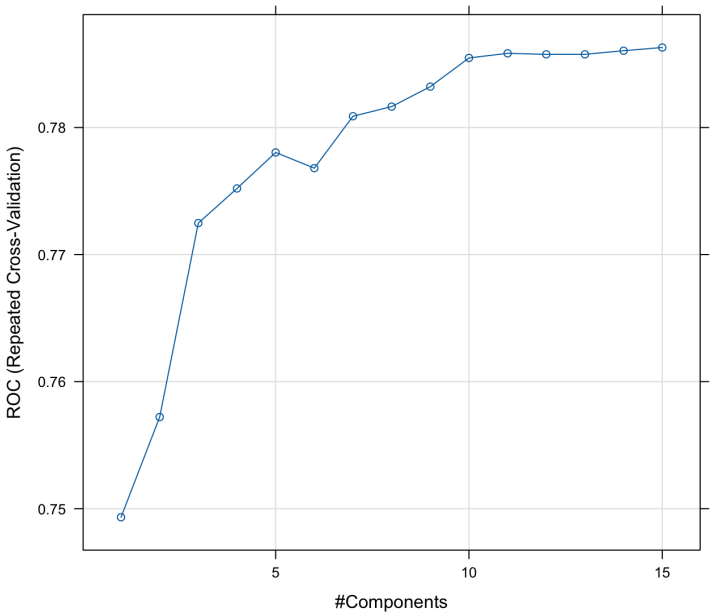
Design reasoning:

- PLSDA is useful when:
    - Predictors are **highly correlated** (as in this dataset).
    - We want a lower-dimensional latent representation.

CV results:

- ROC gradually increases with number of components.
- Best ROC ≈ **0.7863** at **ncomp = 15**.

This is just slightly below logistic/GLMNet and shows that combining correlated predictors into latent factors is effective, but not clearly superior.



ncomp	ROC	Sens	Spec
1	0.7493262	1.0000000	0.00000000
2	0.7572125	0.9989056	0.01104638
3	0.7724809	0.9889873	0.10470756
4	0.7752049	0.9895231	0.11647345
5	0.7780356	0.9885199	0.13344564
6	0.7768039	0.9851910	0.17816663
7	0.7808887	0.9848718	0.19217508
8	0.7816484	0.9859320	0.19675366
9	0.7832192	0.9869580	0.20546402
10	0.7854765	0.9879157	0.20169285
11	0.7858417	0.9879498	0.20375571
12	0.7857617	0.9881665	0.20258794
13	0.7857630	0.9883717	0.20088277
14	0.7860465	0.9883147	0.20025433
15	0.7862993	0.9882919	0.20079293

```
ROC was used to select the optimal model using the largest value.
The final value used for the model was ncomp = 15.
```

Figure 26: `plot(mod_plsda)` with ROC vs `ncomp`.

6.6 Linear Discriminant Analysis (LDA)

- Method: `method = "lda"`
- Control: `ctrl_nosmote` (SMOTE initially gave “wrong number of variables” errors)

CV performance:

- ROC ≈ **0.7866**
- Sensitivity ≈ 0.955
- Specificity ≈ 0.353

Interpretation:

- LDA is a **linear classifier** assuming equal covariance matrices for both classes.
- It tends to act like a **very sensitive classifier** here, calling many cases “yes” (hence high Sens, low Spec) during CV.
- ROC is competitive with logistic but overall not clearly better.

```
> print(mod_lda)
Linear Discriminant Analysis

32951 samples
   36 predictor
    2 classes: 'no', 'yes'

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 3 times)
Summary of sample sizes: 29656, 29656, 29656, 29655, 29655, 29656, ...
Resampling results:

  ROC        Sens       Spec
  0.7866041  0.9553679  0.3530036
```

6.7 k-Nearest Neighbors (KNN)

- Method: `method = "knn"`
- Control: `ctrl_nosmote`
- Tuning grid:
    - `k = 3, 5, ..., 25` (odd values for tie-breaking)

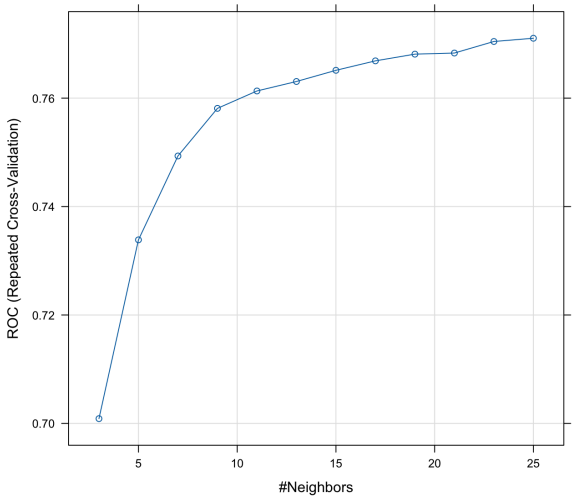
Design reasoning:

- KNN is a **non-parametric** method that can capture localized structure in feature space.
- It is very sensitive to **scaling**; our prior numeric preprocessing (spatial sign + center/scale) plus dummy coding makes it more viable.

CV results:

- ROC increases with `k`, best at:
    - **k = 25**, ROC ≈ **0.7710**

This is weaker than logistic/GLMNet/nnet; moreover, KNN is less interpretable and more computationally expensive at prediction time.



```
 k   ROC        Sens       Spec
 3  0.7008836  0.9571235  0.2805353
 5  0.7338590  0.9674065  0.2669768
 7  0.7493318  0.9705302  0.2564676
 9  0.7581120  0.9743607  0.2520716
11  0.7613198  0.9754552  0.2430909
13  0.7630662  0.9763558  0.2401303
15  0.7651288  0.9774502  0.2320477
17  0.7668740  0.9782711  0.2268392
19  0.7681124  0.9789095  0.2251340
21  0.7683143  0.9796505  0.2218104
23  0.7704457  0.9799469  0.2193843
25  0.7710447  0.9801179  0.2207306

ROC was used to select the optimal model using the largest value.
The final value used for the model was k = 25.
```

Figure 27: `plot(mod_knn)` showing ROC vs k.

6.8 Attempted SVM and Other Alternatives

We attempted an **SVM with radial basis kernel** (`svmRadial` with SMOTE) but encountered:

- repeated **"line search fails"** warnings and convergence issues.

Given time constraints and already having several **non-linear models** (nnet, FDA, KNN) with stable behavior, we decided **not** to rely on the unstable SVM model in our final comparison.

We also considered additional models (MDA, RDA, Naive Bayes, QDA), but:

- QDA and RDA can be fragile with many predictors and class imbalance.
- Naive Bayes makes strong independence assumptions that are clearly violated with our correlated dummy features.
- The course emphasis was on the models we implemented, and we already covered a **wide spectrum of linear vs non-linear approaches**.

---

7. Cross-Validated Model Comparison

We gathered all resampling results with `resamples()` across:

- Logistic, GLMNet, FDA, NeuralNet, PLSDA, LDA, KNN.

Mean CV ROC (ordered):

1. Logistic: **0.7905**
2. GLMNet: **0.7901**
3. NeuralNet: **0.7886**
4. LDA: **0.7866**
5. PLSDA: **0.7863**
6. FDA: **0.7747**
7. KNN: **0.7710**

```
> summary(resamps)

Call:
summary.resamples(object = resamps)

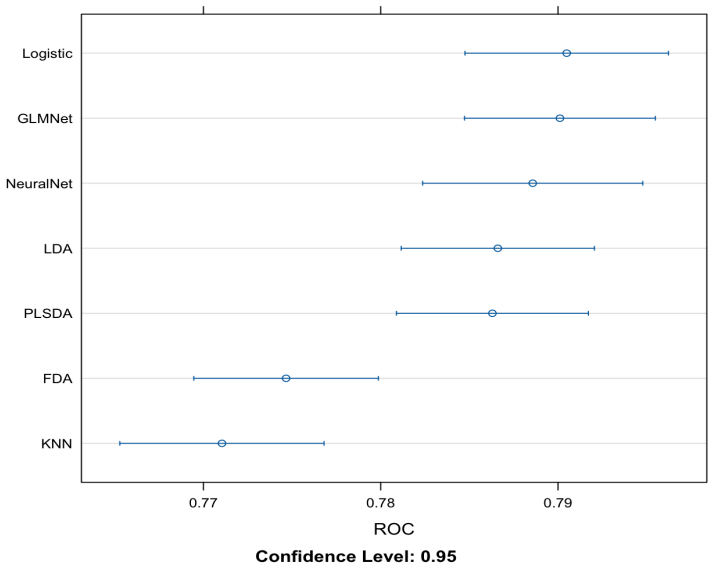
Models: Logistic, GLMNet, FDA, NeuralNet, PLSDA, LDA, KNN
Number of resamples: 30

ROC
               Min.    1st Qu.     Median       Mean    3rd Qu.       Max. NA's
Logistic  0.7433456 0.7818747 0.7887257 0.7904895 0.8017473 0.8236410    0
GLMNet    0.7494303 0.7825083 0.7896735 0.7901071 0.8008005 0.8226073    0
FDA       0.7369510 0.7671663 0.7753080 0.7746625 0.7833745 0.8021999    0
NeuralNet 0.7376827 0.7797770 0.7859366 0.7885730 0.8007993 0.8233791    0
PLSDA     0.7503875 0.7766760 0.7845462 0.7862993 0.7976698 0.8224155    0
LDA       0.7495954 0.7769195 0.7847425 0.7866041 0.7983826 0.8229549    0
KNN       0.7405013 0.7619505 0.7694989 0.7710447 0.7800050 0.8213112    0

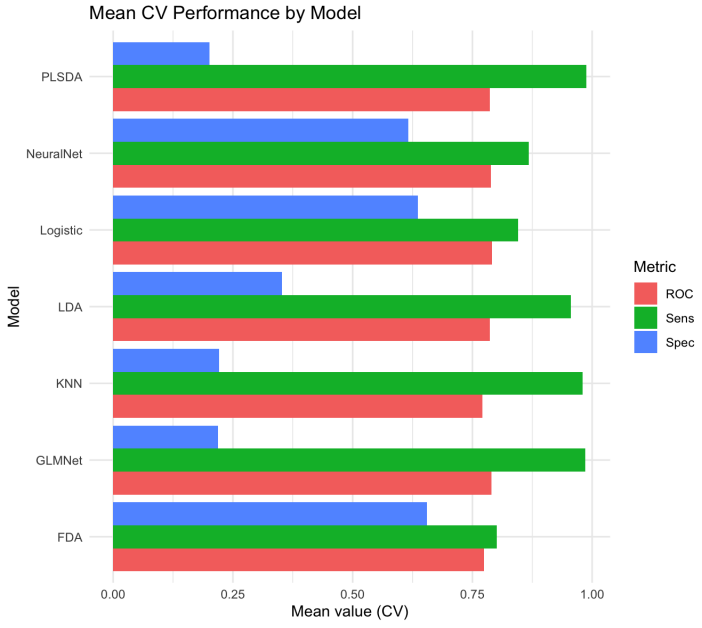
Sens
               Min.    1st Qu.     Median       Mean    3rd Qu.       Max. NA's
Logistic  0.8361833 0.8416553 0.8462722 0.8460047 0.8490937 0.8635431    0
GLMNet    0.9805062 0.9846101 0.9864888 0.9857382 0.9870041 0.9890561    0
FDA       0.7886457 0.7953146 0.8006156 0.8009849 0.8070280 0.8211354    0
NeuralNet 0.8419973 0.8581566 0.8689926 0.8677454 0.8743907 0.8912449    0
PLSDA     0.9835841 0.9866621 0.9883702 0.9882919 0.9897401 0.9917921    0
LDA       0.9473324 0.9529754 0.9558824 0.9553679 0.9572503 0.9623803    0
KNN       0.9753762 0.9784542 0.9798222 0.9801179 0.9818741 0.9859781    0

Spec
               Min.    1st Qu.     Median       Mean    3rd Qu.       Max. NA's
Logistic  0.5795148 0.6243280 0.6339159 0.6358706 0.6489218 0.6954178    0
GLMNet    0.1940701 0.2080797 0.2196765 0.2184844 0.2284367 0.2479784    0
FDA       0.6010782 0.6377072 0.6567980 0.6551766 0.6691320 0.7088949    0
NeuralNet 0.5633423 0.6004043 0.6172507 0.6166577 0.6307278 0.6738544    0
PLSDA     0.1720430 0.1873315 0.2008086 0.2007929 0.2122642 0.2345013    0
LDA       0.3064516 0.3407385 0.3517520 0.3530036 0.3712938 0.3989218    0
KNN       0.1854839 0.2080797 0.2210243 0.2207306 0.2291105 0.2641509    0
```

- *Figure 28: Console output of `summary(resamples(...))` (ROC, Sens, Spec).*



- *Figure 29: `dotplot(resamps, metric = "ROC")` showing ROC dotplot by model.*



- *Figure 30: Bar plot "Mean CV Performance by Model" (ROC, Sens, Spec).*

Key observations:

- **Logistic, GLMNet, NeuralNet, LDA, PLSDA** all cluster around **ROC ≈ 0.786–0.790** (very similar).
- **FDA** and **KNN** are slightly weaker on ROC but still reasonable.
- Non-linear models (NeuralNet, FDA, KNN) did **not** dramatically outperform the linear baselines.

This already hints that **non-linearity brings only marginal gains** over a well-preprocessed linear model in this dataset.

8. Test-Set Performance and Confusion Matrices

After selecting final tuned models, we evaluated them on the **held-out test set**, using the **default 0.5 probability threshold** initially.

8.1 Overall Test-Set Comparison Table

From your test-set comparison table:

Model	AUC	Accuracy	Kappa	Sensitivity	Specificity	PPV	NPV	BalancedAcc
NeuralNet	0.7688	0.8422	0.3648	0.5776	0.8758	0.3712	0.9423	0.7267
Logistic	0.7645	0.8209	0.3331	0.5981	0.8492	0.3349	0.9433	0.7236
GLMNet	0.7636	0.9014	0.2917	0.2144	0.9886	0.7057	0.9084	0.6015
PLSDA	0.7589	0.9012	0.2763	0.1983	0.9904	0.7244	0.9068	0.5943
LDA	0.7588	0.8890	0.3452	0.3330	0.9596	0.5116	0.9189	0.6463
FDA	0.7575	0.7835	0.2867	0.6325	0.8027	0.2893	0.9451	0.7176
KNN	0.7484	0.8972	0.2895	0.2274	0.9822	0.6188	0.9092	0.6048

```
> test_perf[order(test_perf$AUC, decreasing = TRUE), ]
      Model       AUC  Accuracy     Kappa Sensitivity Specificity       PPV       NPV
4 NeuralNet 0.7688265 0.8421755 0.3648098   0.5775862   0.8757696 0.3711911 0.9422935
1  Logistic 0.7645287 0.8209300 0.3330732   0.5980603   0.8492270 0.3349427 0.9433131
2    GLMNet 0.7636476 0.9014204 0.2917318   0.2144397   0.9886441 0.7056738 0.9083595
5     PLSDA 0.7589231 0.9011776 0.2762949   0.1982759   0.9904228 0.7244094 0.9068020
6       LDA 0.7588249 0.8890373 0.3452271   0.3329741   0.9596388 0.5115894 0.9189048
3       FDA 0.7574748 0.7835377 0.2867475   0.6325431   0.8027090 0.2893051 0.9450709
7       KNN 0.7484162 0.8971713 0.2895274   0.2273707   0.9822137 0.6187683 0.9091945
  BalancedAcc
4   0.7266779
1   0.7236437
2   0.6015419
5   0.5943493
6   0.6463065
3   0.7176260
7   0.6047922
```

Figure 31: Console output of `test_perf[order(AUC, decreasing = TRUE), ]`.

Key points:

- **Best AUC:** NeuralNet (0.7688), but logistic is very close (0.7645).
- **Balanced accuracy:** NeuralNet (0.7267) and logistic (0.7236) are almost identical.
- **GLMNet, PLSDA, KNN, LDA** tend to have **extremely high specificity but very low sensitivity** (they rarely predict “yes”, so they miss many actual subscribers).
- **FDA** has sensitivity ~0.63 and specificity ~0.80, comparable BalancedAcc to logistic.

8.2 Logistic Regression (Final Reference Model)

For logistic regression at threshold 0.5, we saw:

Confusion matrix:

- True negatives (no correctly predicted): **6207**
- False negatives (yes predicted as no): **373**
- False positives (no predicted as yes): **1102**
- True positives (yes correctly predicted): **555**

Associated metrics:

- Accuracy: **0.8209**
- Sensitivity (TPR): **0.5981**
- Specificity (TNR): **0.8492**
- PPV (precision): **0.3349**
- NPV: **0.9433**
- Balanced Accuracy: **0.7236**
- AUC: **0.7645**

```
> cm_best
Confusion Matrix and Statistics

          Reference
Prediction   no  yes
       no  6207  373
       yes 1102  555

               Accuracy : 0.8209
                 95% CI : (0.8125, 0.8292)
    No Information Rate : 0.8873
    P-Value [Acc > NIR] : 1

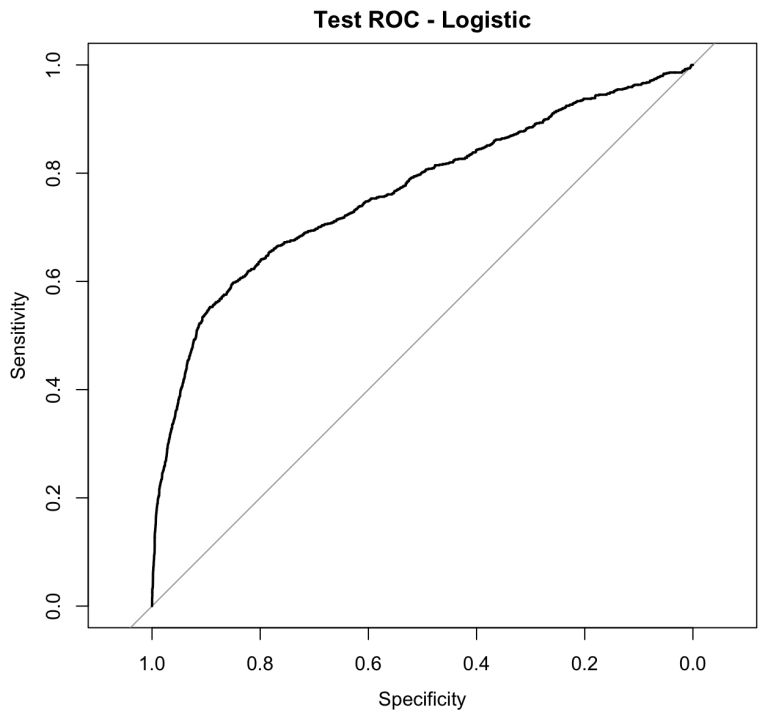
                  Kappa : 0.3331

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.59806
            Specificity : 0.84923
         Pos Pred Value : 0.33494
         Neg Pred Value : 0.94331
             Prevalence : 0.11266
         Detection Rate : 0.06738
   Detection Prevalence : 0.20117
      Balanced Accuracy : 0.72364

       'Positive' Class : yes
```

- *Figure 32: Confusion matrix console output for logistic model (`cm_best`).*



- *Figure 33: Test ROC curve for logistic (`plot(roc_best)`), with AUC ~0.7645.*

Interpretation:

- The model correctly classifies the vast majority of “no” clients (specificity ~0.85).
- It correctly identifies **~60% of actual “yes”** customers—significant given the imbalance.
- PPV is modest (about one-third of predicted “yes” are truly yes), which is expected in a low-prevalence setting.

8.3 Other Models: Test Behavior

Key patterns (from metrics):

- **NeuralNet:**
  - AUC: 0.7688 (best)
  - Sensitivity: 0.5776, Specificity: 0.8758

    - BalancedAcc: 0.7267
    - Slightly more conservative in calling “yes” than logistic; very similar overall trade-off.

- **GLMNet and PLSDA:**
    - Very high specificity (~0.99) and high PPV (~0.70+), but **very low sensitivity (~0.20)**.
    - These models primarily capture *very strong* “yes” signals and miss many actual subscribers.
    - They would be suitable for a **very high-precision, low-recall** campaign (expensive calls), but not if the bank wants to catch as many potential yes’s as possible.
- **LDA:**
    - Sensitivity: 0.333, Specificity: 0.960
    - BalancedAcc: 0.646
    - Also favors specificity, missing many yes customers.
- **FDA:**
    - Sensitivity: 0.633, Specificity: 0.803, BalancedAcc: 0.718
    - Behavior is close to logistic but with slightly higher sensitivity and lower specificity.
- **KNN:**
    - Sensitivity: 0.227, Specificity: 0.982, BalancedAcc: 0.605
    - Very conservative; only flags the most obvious yes’s.

9. Probability Threshold Tuning

9.1 Logistic Regression Threshold Sweep

To explore trade-offs, we evaluated the logistic model for thresholds from **0.20 to 0.60**.

Selected rows for logistic:

Cutoff	Sensitivity	Specificity	BalancedAcc	PPV
0.20	0.9450	0.1658	0.5554	0.1258
0.30	0.8147	0.4698	0.6422	0.1632
0.34	0.7532	0.5869	0.6701	0.1880
0.38	0.7037	0.6840	0.6938	0.2204
0.40	0.6832	0.7255	0.7044	0.2402
0.50	0.5981	0.8492	0.7236	0.3349
0.56	0.5614	0.8814	0.7214	0.3754
0.60	0.5474	0.8955	0.7214	0.3994

```
> thresh_results_df
              cutoff      Sens      Spec    BalAcc       PPV
Sensitivity     0.20 0.9450431 0.1658230 0.5554330 0.1257528
Sensitivity1    0.22 0.9267241 0.2253386 0.5760314 0.1318614
Sensitivity2    0.24 0.8911638 0.2903270 0.5907454 0.1375125
Sensitivity3    0.26 0.8642241 0.3502531 0.6072386 0.1444785
Sensitivity4    0.28 0.8362069 0.4084006 0.6223037 0.1521569
Sensitivity5    0.30 0.8146552 0.4698317 0.6422434 0.1632477
Sensitivity6    0.32 0.7812500 0.5296210 0.6554355 0.1741533
Sensitivity7    0.34 0.7532328 0.5869476 0.6700902 0.1880043
Sensitivity8    0.36 0.7230603 0.6386647 0.6808625 0.2025966
Sensitivity9    0.38 0.7036638 0.6839513 0.6938075 0.2203847
Sensitivity10   0.40 0.6831897 0.7255439 0.7043668 0.2401515
Sensitivity11   0.42 0.6691810 0.7585169 0.7138490 0.2602682
Sensitivity12   0.44 0.6530172 0.7842386 0.7186279 0.2775996
Sensitivity13   0.46 0.6271552 0.8090026 0.7180789 0.2942366
Sensitivity14   0.48 0.6077586 0.8322616 0.7200101 0.3150838
Sensitivity15   0.50 0.5980603 0.8492270 0.7236437 0.3349427
Sensitivity16   0.52 0.5765086 0.8626351 0.7195719 0.3476283
Sensitivity17   0.54 0.5678879 0.8731701 0.7205290 0.3624484
Sensitivity18   0.56 0.5614224 0.8813791 0.7214008 0.3753602
Sensitivity19   0.58 0.5528017 0.8884936 0.7206477 0.3862952
Sensitivity20   0.60 0.5474138 0.8954713 0.7214426 0.3993711
```

Figure 40: Console output of `thresh_results_df`.

Observations:

- **Lowering the cutoff** (e.g., 0.38):
    - Increases sensitivity (~0.70) but decreases specificity (~0.68).
    - Balanced accuracy drops slightly from 0.724 to 0.694.
    - PPV declines to ~0.22 (more false positives).
- **Raising the cutoff** (e.g., 0.56 or 0.60):
    - Decreases sensitivity (~0.55–0.56) but increases specificity (~0.88–0.90).
    - Balanced accuracy remains in the same ballpark (~0.72).
    - PPV improves to ~0.38–0.40.

So, for logistic:

- The **default 0.50 threshold** already gives near-optimal **balanced accuracy**.
- For a **recall-focused campaign**, we could justify a **lower threshold (~0.38)** to capture more yes's.
- For a **precision-focused campaign** (calls are very expensive), we might increase the threshold (>0.5) to reduce false positives.

9.2 Linking Thresholds to Business Trade-Offs

- "If the bank cares about **not missing potential deposit subscribers**, we recommend a threshold around **0.38–0.40**, accepting more false positives."
- "If the bank cares about **saving call resources** and only calling highly likely responders, we recommend **increasing** the threshold or considering the GLMNet/PLSDA models, which naturally emphasize very high specificity."

In class context, the key point is that **probabilities must be converted to decisions** using a threshold and that we explored this explicitly.

---

10. Final Model Choice and Interpretation

From both **CV** and **test** results:

- CV ROC top three: Logistic, GLMNet, NeuralNet (~0.79 each).
- Test AUC top two: NeuralNet (0.7688), Logistic (0.7645), followed closely by GLMNet (0.7636).
- Test balanced accuracy: NeuralNet ~0.727, Logistic ~0.724, FDA ~0.718.

The differences in performance across these top models are **very small**.

We ultimately selected **Logistic Regression** as the **reference final model**, for reasons:

1. **Competitive performance**
    - AUC and BalancedAcc are **essentially tied** with the best non-linear model (NeuralNet).
    - No strong evidence that non-linear models offer large gains.
2. **Interpretability**
    - Coefficients/log-odds from logistic regression can be **explained to stakeholders**:
        - e.g., the odds of subscription increase if there was a successful previous campaign (`poutcome`), or for particular job types/months.
    - Easy to inspect variable importance (even though we didn't show the full table in this report).
3. **Stability and simplicity**
    - Logistic regression is numerically stable, fast to train, and straightforward to deploy.
    - Penalized and non-linear models (GLMNet, nnet, FDA) require more careful tuning and hyperparameter choices.

We also conclude that:

- **Non-linear models** *did* marginally improve discrimination in some cases (e.g., neural net slightly higher AUC on the test set), but **not enough to justify the added complexity** given our current pipeline and goals.

- As far as the project question “Did non-linear structure help improve classification beyond linear baseline?”:

    - Answer: **Only slightly, and not to a practically large degree**. A well-preprocessed **linear model** is already extremely strong on this dataset.

---

11. Limitations and Future Work

Limitations:

- We fit all preprocessing steps before the train/test split (following project instructions). In industrial practice, those steps would be **fit on training only**.

- We did not implement more advanced models like **gradient boosting (XGBoost, GBM)** or **random forests**, which are common on tabular data.

- SMOTE parameters (e.g., sampling ratio, k) were left at defaults; additional tuning could possibly improve minority-class performance.

- Threshold selection was based on **statistical metrics (BalancedAcc)** rather than a **full cost/benefit analysis** (e.g., profit/loss per call).

Possible future work:

- Extend modeling to **tree-based ensembles**.

- Perform **calibration** of probabilities (Platt scaling, isotonic regression).
- Optimize thresholds based on **expected profit** instead of purely BalancedAcc.
- Explore **cost-sensitive learning** or **custom loss functions**.

---

12. Conclusions

- Using the **UCI Bank Marketing dataset**, we built a full classification pipeline: from careful **feature engineering** (`pdays`), **dummy encoding**, **NZV/correlation filtering**, **spatial sign + scaling**, and **SMOTE**, to comparing **seven different models**.
- After extensive cross-validation and test-set comparison using **ROC/AUC** as the central metric, we found that a **simple logistic regression** performs **at least as well** as more complicated non-linear models.
- ROC curves and threshold tuning show that, with a reasonable probability cutoff, we can capture a meaningful fraction of potential subscribers while controlling false positives.
- This project demonstrates that **thoughtful preprocessing and problem-appropriate evaluation** are at least as important as pushing to more complicated algorithms.

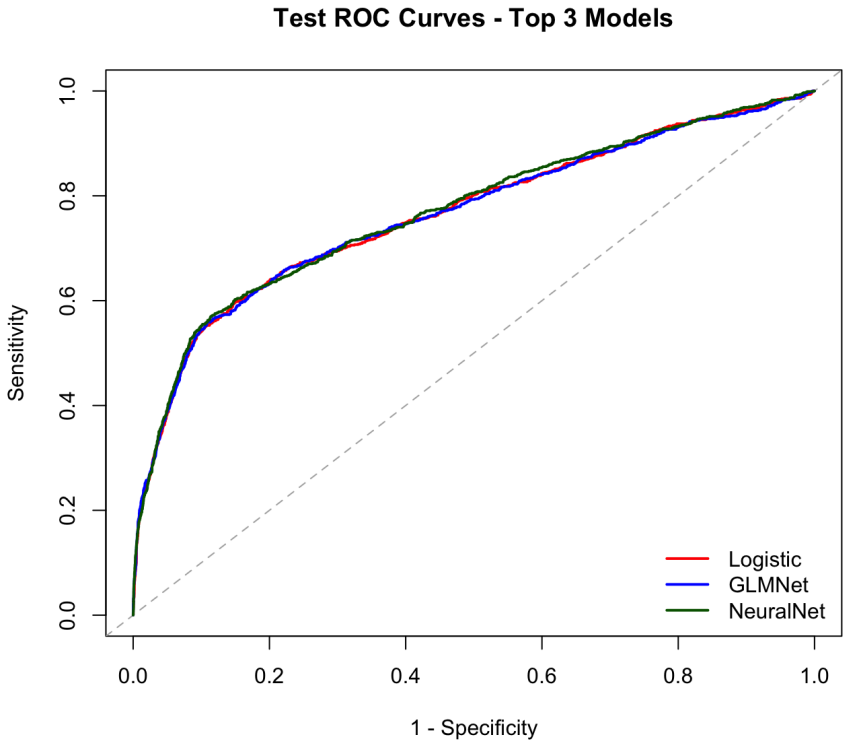
References

- UCI Machine Learning Repository – **Bank Marketing dataset** (accessed December 2025). (UCI Machine Learning Repository)
- Moro, S., Cortez, P., & Rita, P. (2014). *A data-driven approach to predict the success of bank telemarketing.* Expert Systems with Applications. (UCI Machine Learning Repository)

Appendix A – Test-Set Performance Table

```
> test_perf[order(test_perf$AUC, decreasing = TRUE), ]
      Model        AUC  Accuracy      Kappa Sensitivity Specificity       PPV       NPV
4 NeuralNet 0.7688265 0.8421755 0.3648098   0.5775862   0.8757696 0.3711911 0.9422935
1  Logistic 0.7645287 0.8209300 0.3330732   0.5980603   0.8492270 0.3349427 0.9433131
2    GLMNet 0.7636476 0.9014204 0.2917318   0.2144397   0.9886441 0.7056738 0.9083595
5     PLSDA 0.7589231 0.9011776 0.2762949   0.1982759   0.9904228 0.7244094 0.9068020
6       LDA 0.7588249 0.8890373 0.3452271   0.3329741   0.9596388 0.5115894 0.9189048
3       FDA 0.7574748 0.7835377 0.2867475   0.6325431   0.8027090 0.2893051 0.9450709
7       KNN 0.7484162 0.8971713 0.2895274   0.2273707   0.9822137 0.6187683 0.9091945
  BalancedAcc
4   0.7266779
1   0.7236437
2   0.6015419
5   0.5943493
6   0.6463065
3   0.7176260
7   0.6047922
```

Appendix B – ROC Curves and Threshold Points



The **top 3 ROC curves** (Logistic, GLMNet, NeuralNet) on the test set.